

The Secret Language of LLMs

Text becomes tokens before the model processes it

How

1548

are

389

you

527

?

30

text - tokens - token IDs

AI

NAMASTE AI - SEASON 1

Episode 04

The Secret Language of LLMs

Follow a prompt as it becomes tokens and token IDs, then explore subword vocabularies, BPE, multilingual text, emoji and code, special tokens, context windows, and the cost of every extra token.

01:37:44

WebVTT transcript only

In this episode

01 From human text to token IDs**02** Tokenizers, subwords, and vocabularies**03** BPE, WordPiece, and Unigram**04** English, Hindi, Hinglish, emoji, and code**05** Special tokens and hidden message structure**06** Context windows and long conversations**07** Prompt relevance, token cost, and the meaning gap

Which language does an LLM understand?

00:00 - 03:05

Does an LLM understand English? Hindi? Punjabi, Marathi, Telugu, Tamil, Kannada, Malayalam? French or German?

The previous episode said an LLM generates one continuation after another. But before it can continue a prompt, it must receive the prompt in some internal form.

The instructor uses this sentence:

```
Namaste AI is amazing.
```

For a human, `Namaste`, `AI`, and `amazing` trigger meaning. Something “clicks” in the mind. A computer does not directly receive that English-language experience. The same problem exists if the prompt is written in Hindi and the response should continue in Hindi.

The bridge is **numbers**. The lecture's secret language is not one human language. The text is encoded as numbered pieces before the model processes it.

The complete text-to-token pipeline

03:06 – 07:02

The instructor first simplifies `Namaste AI is amazing` into word-like pieces:

```
Namaste | AI | is | amazing
```

Then each piece receives an illustrative number:

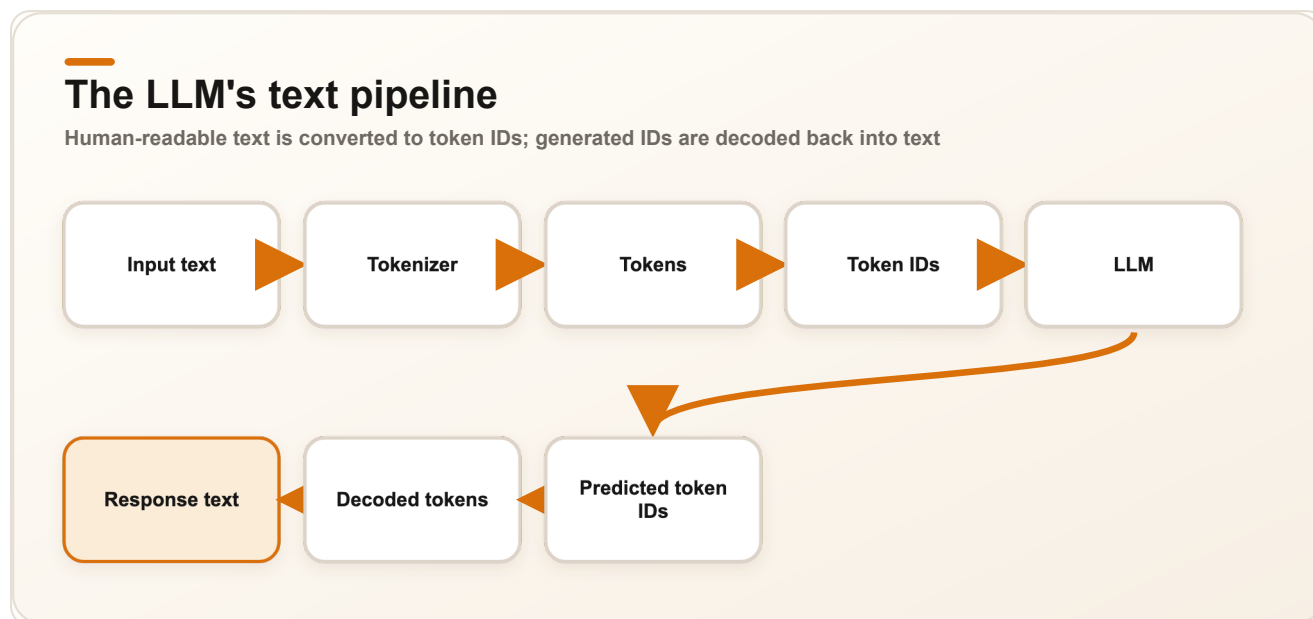
```
Namaste -> 78  
AI       -> 12  
is       -> 37  
amazing  -> 108
```

Those four numbers are invented for explanation. They are not actual IDs from the live tokenizer.

The concepts are:

- A **token** is one piece of the text.
- A **token ID** is the number assigned to that piece in a tokenizer's vocabulary.
- **Encoding** converts text to token IDs.
- **Decoding** converts generated token IDs back into text.

The LLM therefore does not directly predict an English word. In this explanation, it predicts the **next token ID**. Decoding turns that ID sequence back into English, Hindi, code, emoji, or another visible form.



The LLM's text pipeline: encode human-readable text into token IDs, predict IDs, then decode them back into text.

Q: If the LLM sees numbers, how does it know which number comes next?

A: That question will be developed across training and later episodes. For now, the important point is that next-token prediction operates on token IDs, then the tokenizer translates the result for the user.

What is a tokenizer?

07:03 – 10:59

The instructor offers a deliberately simple definition:

A token is a unit of text defined by a tokenizer.

A **tokenizer** is not a mysterious physical machine. It is code: a program, function, or algorithm written by engineers. It accepts a string and breaks/encodes that string into tokens and IDs.

The code runs on a computer, server, CPU, or GPU, but “tokenizer” refers to the algorithm/program itself.

There is no single universal tokenizer:

- OpenAI, Meta, Google, and other model builders can create their own.
- One company can have multiple tokenizers.
- Different model versions can use different tokenizers.
- Tokenizers can improve over time.

Tokenizers are used during both **training** and **inference**. Training data must be encoded for the model; later, user prompts must be encoded in a compatible way, and outputs must be decoded.

One word does not mean one token

11:01 – 18:59

The initial word-by-word split was only an introduction. A tokenizer may split a word into smaller pieces:

```
playing -> play | ing
```

Namaste may also be broken into several parts. That is why the definition says **unit of text**, not **word**.

A token can be:

- a whole common word;
- part of a word;
- one character;
- whitespace combined with a word;
- punctuation;
- all or part of an emoji;
- a piece of code or indentation;
- a special control marker.

The live tokenizer experiment

The instructor opens an online tokenizer and enters:

```
Namaste AI is amazing
```

The selected legacy tokenizer displays:

- **21 characters;**
- **6 tokens.**

The spoken reconstruction of the six colored pieces briefly becomes confused: **AI** is omitted and then added, and the pieces of **Namaste** are transcribed inconsistently. **[Transcript unclear]** The visual frame is needed for an exact selectable boundary list.

Three experiments are stable:

1. Changing lowercase **amazing** to capitalized **Amazing** changes its token ID. The lecture shows **4998** versus **23181** in the selected tokenizer.
2. Removing spaces changes token boundaries and IDs.
3. Selecting a different tokenizer gives different IDs for the same text.

IMPORTANT NOTE

A token ID is local to its tokenizer vocabulary. The number does not have a universal meaning across all models.

The VTT renders the two demo addresses unclearly as `platform.openai.com/...` and `ticktokenizer.vercel.app`. [Incomplete URL] Their exact destinations require the missing video frame.

Why tokenizers use subwords

19:00 – 26:59

The lecture calls the main approach **subword tokenization**.

Common text can stay compact. A simple sentence such as:

```
This is a boy.
```

is shown splitting into familiar word-like tokens.

A less common form can be decomposed into reusable pieces:

```
untrustable -> un | trust | able
```

Why might those pieces be useful?

- `un` appears in many prefixed words.
- `trust` is a common word/piece.
- `able` appears in many other forms.

The tokenizer tries to reuse pieces instead of creating a unique token for every full word.

Extreme 1: one token per whole word

This keeps the sequence short, but the vocabulary becomes enormous. It must contain `trust`, `untrust`, `untrustable`, and countless other complete variations.

Extreme 2: one token per character

This keeps the vocabulary small. In a simplified English-only example, the 26 letters could represent every word. But sequences become long:

```
untrustable -> 11 character tokens
```

Longer token arrays require more mathematical work inside the model.

The subword balance

Whole words

Short sequences, but a very large vocabulary of complete forms.

Characters

Small vocabulary, but long sequences and more computation.

Subwords aim for a sweet spot: a manageable vocabulary plus sequences that are not unnecessarily long.

Vocabulary and token IDs

27:02 – 32:59

A tokenizer's **vocabulary** is the collection of tokens it knows how to represent directly, together with their ID mapping.

The instructor sketches hypothetical mappings such as:

```
un    -> 373
trust -> 90349
able  -> 562
the   -> 10
ing   -> 15
.     -> 14
```

These values illustrate the mapping idea; they are not presented as a verified vocabulary table.

Different tokenizers have different vocabulary sizes. The lecture mentions OpenAI tokenizer families with roughly **50K** and **200K** tokens and speaks one count as approximately **50,257**.

The exact encoding names are badly transcribed into phrases such as “our 50 K base,” “peak 50 K base,” and “O 200 K base.” [Identifier unclear in transcript]

Vocabulary size describes how many token pieces can be represented directly. A tokenizer with around 200,000 entries has roughly 200,000 reusable pieces available when it encodes text.

Other providers have their own tokenizers, vocabulary sizes, and encoding/decoding rules.

Byte-pair encoding (BPE)

33:01 – 39:59

The instructor says OpenAI tokenizers commonly use **byte-pair encoding**, or **BPE**, then gives a deliberately short explanation.

Consider:

low
lower
lowest

The sequence **l** followed by **o** appears repeatedly. Merge that frequent neighboring pair into **lo**. If **lo** followed by **w** is also frequent, merge it into **low**. Add each reusable merged piece to the vocabulary with an ID.

The compact definition is:

Byte-pair encoding builds a vocabulary by repeatedly merging frequently occurring neighboring pieces.

A five-minute BPE sketch

Frequently recurring adjacent pieces are merged into reusable vocabulary tokens



The lecture's five-minute BPE sketch: repeatedly merge neighboring pieces that occur frequently.

The instructor briefly connects the name to bits and bytes:

- a bit is **0** or **1**;
- a byte contains eight bits;
- characters can be represented numerically;
- recurring adjacent pieces can be paired and assigned a new vocabulary ID.

The bit patterns and numbers drawn are illustrative. The instructor says there are “255 combinations,” but the transcript does not establish that number as a formal binary lesson.

The purpose of this detour is curiosity, not implementation. A complete BPE algorithm would require a separate lesson.

WordPiece and Unigram: two brief sketches

40:01 – 42:59

BPE is not the only way to create a tokenizer vocabulary.

WordPiece

WordPiece is described as selecting useful vocabulary pieces according to how well they represent training data, often with a longest-match-first style. `playing` is used as an example of a base piece plus an ending piece.

The instructor explicitly says he has **not researched WordPiece deeply** and does not fully understand it in this lecture. The precise ending marker is also unclear in the transcript. `[Transcript unclear]`

Unigram

Unigram is presented as moving in the opposite direction from BPE:

1. Begin with a large set of possible pieces.
2. Identify pieces that are less useful.
3. Remove them.
4. Represent rare forms through smaller reusable pieces.

A rare full token such as `untrustable` could be removed while `un`, `trust`, and `able` remain.

THE SAFE COMMON IDEA

Modern tokenizers break unfamiliar text into reusable subword units instead of requiring every complete word to be a unique vocabulary entry.

Familiar English versus gibberish

43:00 – 47:59

The live tool compares an ordinary English sentence with a similar-length gibberish string.

- The English sentence uses **14 tokens**.
- The gibberish portion is described as **38 tokens**.
- The combined display reaches **51 tokens**.
- An older GPT-3/legacy selection reaches **65 tokens** for the demonstrated combined content.

The exact gibberish string and complete English sentence are not recoverable from the WebVTT.

`[Video frame needed]`

Why the difference? Common English patterns already exist as larger vocabulary pieces. Random character sequences match fewer pieces and must be broken down more finely.

Newer/richer vocabularies can represent the same text with fewer tokens. But compact tokenization does not attach a dictionary definition to each token.

TOKEN BOUNDARIES ARE NOT MEANING BOUNDARIES

If `India` becomes one token ID, the tokenizer has performed a string-to-number mapping. The semantic relationship of India to other concepts has not yet been explained.

English, Hindi, mixed script, and Hinglish

48:00 – 55:59

The instructor enters the same idea-roughly “I am learning artificial intelligence”-in three forms:

- English;
- Hindi script;
- a mixed English/Hindi form.

They express the same meaning but produce different token counts.

Counts shown in the newest tokenizer

Form	Tokens
English	6
Hindi	15
Mixed script	10

The Hindi line uses **68 tokens** in the GPT-3/legacy choice and **44** in an intermediate choice, compared with **15** in the newest selection. For a larger combined multilingual sample, the lecture reports **118**, **83**, and **39** tokens across successively newer tokenizer choices.

The exact Devanagari sentences are not captured reliably in the VTT. [Exact source text unavailable]

The instructor reads the improvement as richer Hindi vocabulary coverage in newer tokenizers. The lecture does not establish that fewer tokens alone measure a model's entire language ability.

Why Hinglish is especially interesting

Hinglish can mix:

- English vocabulary;
- Hindi grammar;
- Roman script;
- informal spellings;
- language-specific expressions.

The same intended word can be typed as `main`, `mai`, or another variant. `Samjhao` can also appear in several spellings. Humans may understand the shared intention, but the tokenizer sees different character strings and can assign different token sequences.

IMPORTANT NOTE

Two sentences with the same meaning do not necessarily have the same token count-or any shared token IDs.

Tokenization fertility

55:00 – 56:59

The instructor introduces the phrase **tokenization fertility** because he encountered it while researching the lesson.

It measures how many tokens are produced for a word, character sequence, or linguistic unit. Higher fertility means the text has been divided into more pieces.

This is a label for the result of tokenization, not a new model process. The instructor says the term may not be especially relevant later, but it gives us vocabulary for comparing compact and fragmented encodings.

Emoji are text too

56:02 – 58:59

The tokenizer must handle anything a user can type, including emoji.

The live examples show:

- a mind-blown emoji split into two tokens;
- a heart represented as one token;
- fire represented as one token;
- laughter/face emoji split differently;
- a sweat-smile emoji remaining two tokens even in the newer tokenizer.

Newer tokenizers represent more of the displayed emoji compactly than older selections. The exact emoji row and comparison counts are not preserved in the captions. [Video frame needed]

The general rule is:

One emoji is not necessarily one token.

Whitespace, case, punctuation, and code

58:59 – 01:03:59

Tokenization reacts to formatting, not only letters.

Whitespace

`Hello world` is shown as two tokens. Adding spaces or a tab changes the sequence. A leading space can be part of the token that contains the following word.

Capitalization

`hello`, `Hello`, and an all-capital form can produce different IDs and even different boundaries.

Reusable prefixes and endings

The lecture experiments with:

```
untrustable  
unconditionable  
unimaginable
```

Pieces such as `un` and `able` can recur while the middle stem is tokenized differently. Several captions in this live segment are corrupted, but the recurring-prefix/suffix point is clear.

Punctuation

A comma and question mark have token representations. Inserting a space before punctuation can change the token IDs.

Code

Keywords, braces, punctuation, newlines, indentation, tabs, and spaces all participate in tokenization. Moving code to the next line or changing its indentation changes the token sequence.

The exact code entered in the demo is not recoverable. `[Exact code was not visible in the supplied material]`

WHY GENERATED CODE HAS FORMATTING

The model has been trained on token patterns that include newlines and indentation. Formatting is part of the represented sequence, not an afterthought outside the model.

The prompt contains more than what you type

01:04:01 - 01:08:01

Suppose the visible prompt is:

```
How are you?
```

The public tokenizer may show only the ordinary text tokens. But the complete assistant request includes more structure.

Possible hidden/control information named in the lecture includes:

- system instructions;
- start and end of a message;
- user and assistant roles;
- tool-output boundaries;
- code, document, or quoted-string markers;
- reserved/special tokens.

The instructor sketches a conversation as `system`, `user`, and `assistant` blocks. Each boundary could be encoded through special token IDs.

DO NOT MISTAKE THE SKETCH FOR AN EXACT WIRE FORMAT

The lecture says serialization varies by provider, model, and system. The model may receive special IDs rather than visible labels, and only the provider knows the exact final text/token sequence.

Context: why “here” means Dehradun

01:08:02 – 01:15:59

The instructor starts a temporary ChatGPT conversation:

```
Hello, I am in Dehradun right now.
```

Then asks:

```
Which places can I visit here?
```

The model understands `here` as Dehradun because the earlier message is supplied as context.

It returns place suggestions. A later instruction to sort “these places” also depends on the previous response. If the user then asks whether to carry an umbrella, the assistant can use a weather tool's earlier output.

Context may contain:

- system instructions;
- the current user prompt;
- previous user and assistant messages;
- uploaded PDF/document text;
- retrieved information;
- tool output such as weather;
- the model's earlier generated responses.

All of this consumes tokens.

The lecture defines a **context window** as:

The amount of tokenized information a model can process within a request or active generation context.

It is a finite working window, not unlimited memory.

Input and output share the context window

01:16:01 – 01:18:59

The context limit includes both what is sent and what the model generates.

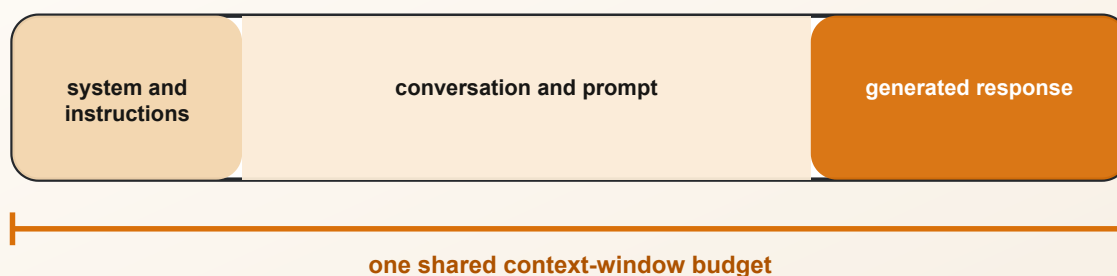
The instructor's numerical example is:

```
Total context: 10,000 tokens
Input:         8,000 tokens
Output space:  2,000 tokens
```

If the input uses only 2,000 tokens, then 8,000 remain in this simplified model. Modern assistants may reserve output space or impose separate product limits, but the theoretical point is a shared budget.

The context window is shared

Input, conversation history, instructions, and generated output all consume the available window



Instructions, conversation history, current input, retrieved/tool material, and generated output all consume one finite context-window budget.

Every processed or generated token requires compute. The instructor therefore connects token count with the expense of public ChatGPT access. He calls OpenAI loss-making and suggests free access helps build user habit; those are time-bound business claims from the lecture, not verified here.

What happens when the window fills?

01:19:00 – 01:23:59

Different applications can handle overflow differently. The instructor lists possible strategies:

- reject the request;
- truncate part of the input;
- remove older conversation turns;
- summarize older messages and keep the summary;
- retrieve only relevant sections, connected here with RAG;
- ask the user to shorten the input;

- split the work into multiple requests.

He explicitly says he has not deeply researched exactly what ChatGPT does internally. Truncation and summarization are therefore presented as likely strategies, not confirmed implementation details.

Imagine a chat that begins with Dehradun, then moves through football, cricket, JavaScript, Twitter, Node.js, and artificial intelligence. After 100 or 200 messages, `Which places can I visit here?` may no longer connect `here` to Dehradun because that old detail has moved out of active context.

VISIBLE CHAT ≠ ACTIVE CONTEXT

The interface can display 100 old turns from a database while the current model request contains only a subset or summary. Seeing a message on screen does not prove the model is processing it now.

Context limits can also differ by model and product plan. The lecture gives no exact sizes and its Free/Pro comparison is time-bound.

Long prompt does not mean better prompt

01:24:02 – 01:27:59

The instructor compares three JavaScript-closure prompts.

Short and sufficient

`Explain closures in JavaScript with one simple example.`

Long but repetitive

The second version repeatedly asks for an understandable, uncomplicated, beginner-friendly explanation and an easy example. It is longer but does not materially change the task.

Long and relevant

`Explain closures in JavaScript to a beginner who understands functions and scope but has never seen lexical environments. Use one example involving a counter. Keep it under 250 words.`

Every added detail matters:

- `beginner` defines the audience;
- `understands functions and scope` defines prior knowledge;
- `never seen lexical environments` identifies the missing concept;
- `one example involving a counter` specifies the demonstration;
- `under 250 words` limits the output.

Q: Is a long prompt bad?

A: Not necessarily. A long prompt is useful when its extra tokens add relevant context or constraints. Repetition alone adds cost, not clarity.

The instructor compares the redundant prompt with filling examination pages when we do not have more substance to add. In an LLM request, remove filler and retain information that changes the desired response.

Tokens and API cost

01:26:00 – 01:27:59

The lecture states that API billing depends on token use. More input and output tokens produce more cost; fewer tokens produce less cost.

That makes prompt relevance practical, not merely stylistic:

- remove redundant input;
- retain necessary constraints;
- bound the output when appropriate;
- remember that generated tokens also count.

No exact API price is supplied in the source material.

Five misconceptions to remove

01:28:00 – 01:31:00

1. One token equals one word

False. A token may be a subword, character, whitespace-plus-word, punctuation, emoji piece, code piece, or special marker.

2. One emoji equals one token

Not necessarily. Some emoji are one token; others split into several.

3. Larger vocabulary is always better

Not necessarily. A coding-focused model or a model built for a particular language can benefit from a vocabulary designed for that use. A generic model needs a different balance.

4. Larger context means perfect memory

False. It allows more tokenized information, but it remains finite and comes with limits.

5. More tokens always produce better results

False. More relevant information can help; more filler does not.

Tokens explain the language-but not the meaning

01:31:01 – 01:34:59

The episode now has a complete representation pipeline:

- 1 The user writes a prompt.
- 2 A tokenizer divides it into ordinary and special tokens.
- 3 Each token maps to an ID in that tokenizer's vocabulary.
- 4 The LLM processes the ID sequence and predicts further IDs.
- 5 The tokenizer decodes those IDs into the visible response.

Text, code, whitespace, punctuation, and emoji all follow this broad route. Different tokenizers can choose different boundaries and IDs.

This is the secret language promised in the title: **the language of tokens**.

But one question remains. Suppose the lecture invents arbitrary IDs:

```
dog      -> 12
cat      -> 32
cow      -> 07
programming -> 9
coding   -> 17
Python   -> 34
```

How does the LLM know that dog, cat, and cow are related as animals? How does it connect programming, coding, JavaScript, Python, Ruby, and React? How does it group cricket, basketball, tennis, and football as sports? How does it relate Delhi, Mumbai, and India to places?

Context makes the ambiguity even richer. `Python` can refer to programming or a snake/reptile. Java and JavaScript have related names but are different. Token IDs alone do not explain those relationships.

Practice and the next question

01:35:03 – 01:37:44

The instructor asks learners to open an online tokenizer and experiment:

- compare older and newer model tokenizers;
- type common words and unfamiliar strings;
- try English, Hindi, and mixed script;
- change capitalization and whitespace;
- paste emoji;

- paste code and change indentation;
- compare token counts across model families, including Llama and a Google model transcribed as `GAMA` in the second tool. `[Identifier unclear in transcript]`

The goal is not to memorize IDs. It is to see that the same visible meaning can produce different token sequences, and tiny formatting changes can alter the encoding.

The next episode introduces **embeddings**. It will address the unresolved question: if token IDs are arbitrary numbers, how does the model represent that concepts are similar, related, or context-dependent?

Chapter summary

An LLM does not directly consume English, Hindi, emoji, or code as humans see it. A tokenizer encodes the input into token IDs drawn from a particular vocabulary. The LLM predicts further IDs, and the tokenizer decodes them into a response.

Tokens are reusable text pieces, not necessarily words or characters. Subword tokenization balances vocabulary size and sequence length. BPE grows a vocabulary by repeatedly merging frequent neighboring pieces; WordPiece and Unigram are only sketched at a high level.

Tokenization depends on familiarity, language coverage, whitespace, case, punctuation, emoji, and code formatting. The visible prompt is surrounded by system, role, message, tool, and document structure. Together with prior conversation and retrieved information, these tokens occupy a finite context window shared with generated output.

Prompt length matters only when the extra tokens add useful information. Token count affects API cost. And after all that, tokens still do not explain meaning—the bridge to embeddings in the next episode.

Key takeaways

- A tokenizer encodes human-readable text into token IDs and decodes generated IDs back into text.
- One token is not the same as one word or one character.
- Token IDs are meaningful only inside the vocabulary that assigns them.
- Subwords balance whole-word vocabulary size against character-level sequence length.
- BPE is introduced as repeatedly merging frequent neighboring pieces.
- Token boundaries reflect reusable strings, not semantic definitions.
- The same meaning can have different token counts across English, Hindi, mixed script, and spelling variants.
- Emoji, whitespace, case, punctuation, indentation, and code can all change tokenization.
- The visible prompt is only one part of the model context; special tokens, history, documents, retrieval, and tools may also be present.
- A context window is finite and shared by input and output.
- Long prompts help only when their added tokens supply relevant context or constraints.
- Tokenization explains representation, while embeddings are left to explain meaning.

Revision questions

1. Why does the instructor call tokens the LLM's secret language?
2. Trace the encode-predict-decode pipeline from prompt to response.
3. Define token, token ID, tokenizer, encoding, and decoding.
4. Why is `playing -> play | ing` an important correction to the word-level explanation?
5. What three changes in the live demo alter token IDs?
6. Why can token IDs not be compared freely across tokenizers?
7. What is subword tokenization?
8. What disadvantage comes with a whole-word vocabulary?
9. What disadvantage comes with character-only tokenization?
10. How does `untrustable -> un | trust | able` illustrate reuse?
11. What does a tokenizer vocabulary contain?
12. How does the lecture explain BPE with `low`, `lower`, and `lowest`?
13. What limitation does the instructor attach to his WordPiece explanation?
14. How does the Unigram sketch differ from the BPE sketch?
15. Why does gibberish usually consume more tokens in the demonstration?
16. Explain “token boundaries are not meaning boundaries.”
17. What do the English, Hindi, and mixed-script counts show?
18. Why is Hinglish difficult for a tokenizer?
19. What does tokenization fertility measure?
20. Why can one emoji become more than one token?

21. How do whitespace and capitalization affect token IDs?
22. Why does code indentation participate in tokenization?
23. Which special/control information may surround the visible prompt?
24. Why is the `system/user/assistant` sketch not an exact provider format?
25. What does the Dehradun example teach about context?
26. Which items can occupy a context window?
27. Why do input and output share one context budget in the lecture's model?
28. What can happen when the context window fills?
29. Why can visible conversation history differ from active model context?
30. Compare the short, redundant, and relevant JavaScript-closure prompts.
31. How does token use affect API cost?
32. Which five misconceptions does the episode reject?
33. What meaning question remains unanswered after tokenization?
34. Which tokenizer experiments does the instructor ask learners to perform?